



KCA UNIVERSITY
SYSTEM REQUIREMENTS SPECIFICATION
SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title:

Predicting Developer Technology Decline Using Community Signals and Enterprise Adoption Patterns: A Machine Learning Approach

System Name: TechPulse

Student Name	Kitili Tony Kenga
Registration Number	24/03652
ORCID	0009-0007-6899-8590
Course Title	Final Year Project I
Course Code	STU 4101
Programme	Bachelor of Science in Data Science
Supervisor	Dr. Rufus Gireka
Faculty	School of Technology (SoT)
Date Submitted	27th June 2026
Document Version	1.0

Table of Contents

Contents

Table of Contents	2
4.1 Introduction.....	5
4.1.1 Purpose.....	5
4.1.2 Scope.....	6
4.1.3 Acronyms and Abbreviations	7
4.2 Overall Description	9
4.2.1 System Perspective	9
4.2.2 System Environment.....	10
Hardware Requirements.....	10
Software Requirements	10
Development Tools	10
Python Libraries and Frameworks	10
Database.....	11
Network and Internet Requirements	11
Cloud Services	11
4.2.3 User Characteristics	12
User Role 1: Technology Decision-Makers (Primary Users).....	12
User Role 2: Individual Developers (Primary Users)	12
User Role 3: Academic Researchers and Practitioners (Secondary Users)	12
User Role 4: Project Supervisor (Administrative User)	13
4.2.4 Constraints	13
4.3 Functional Requirements	15
FR-01 Dataset Ingestion and Validation.....	15
FR-02 Exploratory Data Analysis (EDA).....	16
FR-03 Feature Engineering.....	17
FR-04 Target Variable Assignment	18
FR-05 Model Training Logistic Regression Baseline	19
FR-06 Model Training K-Nearest Neighbours Baseline	20
FR-07 Data Splitting and Cross-Validation Setup.....	21
FR-08 Model Training Random Forest.....	22
FR-09 Model Training XGBoost	23
FR-10 Model Evaluation and Comparison	24
FR-11 SHAP Feature Importance Analysis	25
FR-12 Dashboard Technology Search and Filtering	26

FR-13 Dashboard Technology Risk Score and Trajectory Prediction Display	27
FR-14 Dashboard Trend Charts and Historical Signal Visualisation	28
FR-15 Dashboard SHAP Feature Importance Viewer	29
FR-16 Dashboard Enterprise Adoption Heatmap	30
FR-17 Dashboard Global Technology Risk Rankings	31
FR-18 Dashboard Model Performance Summary Page	32
FR-19 Dashboard Disclaimer and Documentation Page	33
FR-20 Dissertation Documentation	33
4.3.1 Use Case Diagram Description	34
Actors	35
Major Use Cases	35
4.4 Non-Functional Requirements	37
4.4.1 Performance	37
4.4.2 Scalability	37
4.4.3 Security	38
4.4.4 Availability	38
4.4.5 Reliability	38
4.4.6 Maintainability	39
4.4.7 Usability	39
4.4.8 Data Integrity	40
4.4.9 Privacy and Compliance	40
4.4.10 Auditability and Logging	41
4.5 External Requirements	42
4.5.1 User Interfaces	42
UI-01: Dashboard Home Page / Technology Search Page	42
UI-02: Technology Detail Panel	42
UI-03: Global Risk Rankings Page	42
UI-04: Model Performance Page	43
UI-05: About / Documentation Page	43
4.5.2 Software Interfaces	43
PostgreSQL 14+ Database Interface	43
Streamlit Framework Interface	44
Streamlit Cloud Deployment Interface	44
GitHub Version Control Interface	44
SHAP Library Interface	44
4.5.3 Hardware Interfaces	44
Development Computer	44

End-User Device	45
Network Hardware.....	45
References.....	46

4.1 Introduction

4.1.1 Purpose

This Software Requirements Specification (SRS) document formally defines the functional, non-functional, and external requirements for TechPulse, a machine learning-based technology trajectory prediction system. The document is intended to serve as the authoritative reference for all design, development, and evaluation activities undertaken as part of the Final Year Project STU 4101 in the Bachelor of Science in Data Science programme. It provides a comprehensive and unambiguous specification of what the system must accomplish, the constraints under which it must operate, and the quality standards it must meet.

The business problem motivating TechPulse is one with measurable financial and professional consequences. Technology adoption decisions whether made by organisations selecting components for their software stacks or by individual developers choosing where to invest their learning effort carry significant risk. An organisation that commits engineering resources to a technology that enters decline faces expensive, disruptive migrations. A developer who deepens specialisation in a declining technology faces narrowing job prospects and forced reskilling. Despite these high stakes, the existing decision-making landscape is dominated by informal channels: trend articles, social media commentary, and peer opinion rather than rigorous, data-driven analysis. The result is that consequential decisions are routinely made on the basis of lagging or anecdotal evidence.

TechPulse addresses this gap by extending an existing open-source analytics repository developer-ecosystem-analytics (19 GitHub stars) with a supervised machine learning layer. The existing repository aggregates six real-world datasets covering Stack Overflow community activity, developer sentiment surveys, Fortune 500 technology adoption patterns, company profiles, technology metadata, and question-to-company mappings. It implements twelve descriptive SQL analytics queries and computes a composite Technology Health Score. What it does not yet offer is forward-looking, predictive capability. TechPulse adds that capability by engineering predictive features from the existing datasets, training and evaluating classification models, and surfacing predictions through an interactive public dashboard.

The intended users of TechPulse are: technology decision-makers within organisations (including chief technology officers, engineering managers, and solution architects) who require evidence-based input for stack selection and risk mitigation; individual developers seeking data-driven guidance on which technologies are worth investing in; and academic

researchers and practitioners in the field of software analytics who may use or extend the published findings. The system is also expected to benefit the broader open-source developer community, since TechPulse will be released as a public extension of the existing starred repository.

The expected outcomes of this project are: a trained machine learning model achieving a target accuracy of at least 80 percent on a three-class technology trajectory classification task (growing, stable, declining); a SHAP-derived ranking of the top predictors of technology decline; a publicly deployed interactive Streamlit dashboard providing real-time risk scores and predictions for more than 50 technologies; a formal dissertation meeting all BSc. Data Science requirements; and a potential conference or journal submission based on the findings. These outcomes collectively constitute an original research contribution that extends descriptive community analytics into the domain of predictive, explainable machine learning applied to technology ecosystem health.

4.1.2 Scope

TechPulse is a machine learning research and prediction system. The scope of the system is defined precisely below in order to ensure that project boundaries are unambiguous and that the work plan can be executed within the two-semester academic timeline.

The system WILL accomplish the following: ingest and process the six datasets already integrated in the developer-ecosystem-analytics PostgreSQL database without requiring new data collection; perform comprehensive exploratory data analysis (EDA) across all six datasets, including statistical correlation analysis and distribution analysis; engineer a set of seven predictive features from existing SQL query outputs and raw dataset fields, including Technology Health Score, Growth Momentum Index, Question Quality Score, Company Diversity Score, Sentiment Delta, Adoption Velocity, and Community Decay Rate; assign target class labels (growing, stable, or declining) to technologies based on 12-month trailing data; train and evaluate four classification models Logistic Regression, K-Nearest Neighbours, Random Forest, and XGBoost with hyperparameter tuning for the advanced models; evaluate model performance using accuracy, weighted F1-score, ROC-AUC (where applicable), and per-class confusion matrix analysis with 5-fold cross-validation; generate SHAP feature importance values for the best-performing model; build an interactive Streamlit dashboard that displays technology risk scores, trajectory predictions, trend charts, enterprise adoption

heatmaps, and SHAP feature importance visualisations; deploy the dashboard publicly via Streamlit Cloud; and document all findings, methodology, and recommendations in a formal dissertation.

The system WILL NOT accomplish the following: collect new primary data beyond the six datasets already integrated in the developer-ecosystem-analytics repository; provide regulated financial or investment advice; guarantee prediction accuracy for technologies with insufficient historical signal data, which will instead be flagged with a low-confidence indicator; monitor technology signals in real time updates will be delivered through scheduled batch processing rather than live streaming; support multi-language interfaces or localisation beyond English; process personal or private data about individual developers; integrate with paid third-party APIs or cloud services beyond the free tier of Streamlit Cloud; or train deep learning models, as the project is scoped to classical supervised machine learning methods on structured tabular data.

System limitations that must be acknowledged include: prediction accuracy is bounded by the quality, recency, and completeness of the six source datasets; class imbalance between growing, stable, and declining technologies may require mitigation strategies such as stratified sampling; technologies with fewer than a defined minimum number of data points across the source datasets will be excluded from prediction to protect result validity; and the Streamlit Cloud free tier imposes resource constraints on the deployed dashboard that may affect responsiveness under high concurrent load.

Future expansion possibilities beyond the current project scope include: integration of real-time Stack Overflow API data feeds to enable live signal monitoring; extension of the model to cover additional technology categories; integration of GitHub repository activity signals as additional predictive features; development of a REST API layer to allow programmatic access to TechPulse predictions; and multi-language support for the dashboard.

4.1.3 Acronyms and Abbreviations

Acronym / Term	Full Meaning
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve

Acronym / Term	Full Meaning
BSc.	Bachelor of Science
CSV	Comma-Separated Values
DB	Database
EDA	Exploratory Data Analysis
F1	F1-Score (harmonic mean of precision and recall)
FR	Functional Requirement
GH	GitHub
HTTP/HTTPS	HyperText Transfer Protocol (Secure)
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbours
LR	Logistic Regression
ML	Machine Learning
NFR	Non-Functional Requirement
ORCID	Open Researcher and Contributor ID
PGSQL / SQL	PostgreSQL / Structured Query Language
REST	Representational State Transfer
RF	Random Forest
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SO	Stack Overflow
SRS	Software Requirements Specification
STU	Student (course code prefix)
TechPulse	The system being specified in this document
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
XGBoost	Extreme Gradient Boosting

4.2 Overall Description

4.2.1 System Perspective

TechPulse is not a standalone, greenfield system. It is designed explicitly as a predictive extension of the existing developer-ecosystem-analytics open-source repository, which provides the data infrastructure and descriptive analytics layer that TechPulse builds upon. The system therefore occupies a position at the boundary between data engineering, machine learning research, and interactive software deployment.

From an architectural perspective, TechPulse can be divided into four conceptual layers. The first is the Data Layer, which consists of the six pre-integrated datasets stored in a PostgreSQL 14+ relational database. This layer is pre-existing and is consumed but not structurally modified by TechPulse. The second is the Feature Engineering and Modelling Layer, implemented in Python, which reads from the database, computes the predictive feature matrix, assigns target labels, and trains and evaluates the four classification models. The third is the Explainability Layer, which applies SHAP analysis to the best-performing trained model to derive feature importance values and per-prediction explanations. The fourth is the Presentation Layer, which is the Streamlit dashboard a Python web application that loads the trained model and SHAP values, provides a user interface for technology search and filtering, and visualises risk scores, trajectory predictions, trend charts, and feature importance for each queried technology.

The system interacts with the following external components: the PostgreSQL database (read access only for TechPulse); the Streamlit Cloud hosting platform, which provides the public deployment environment for the dashboard; and GitHub, which serves as the version control and collaboration platform for the project codebase. No other external services are required for the core system. Optional future integrations (Stack Overflow API, GitHub API) are noted in the scope section but are not part of the current project.

A block diagram description of the system architecture is as follows: at the bottom layer, the PostgreSQL database contains six tables corresponding to the six source datasets (Stack Overflow Questions, Developer Sentiment Survey, Fortune 500 Tech Stacks, Company Profiles, Technology Metadata, and Question-Company Mapping). Above this sits a Python data pipeline that reads from the database via SQL queries (reusing and extending the 12 existing queries), computes the feature matrix, assigns labels, and writes the feature matrix to a CSV file. The ML Training Module reads the feature matrix, splits data 80/20 with stratified sampling, applies 5-fold cross-validation, trains four classifiers, and evaluates them. The best

model is serialised to disk (pickle/joblib format) along with a SHAP explainer object. The Streamlit Dashboard Application loads the serialised model and explainer at startup and serves an interactive web UI accessible via public URL on Streamlit Cloud. End users interact exclusively through the web browser interface.

4.2.2 System Environment

Hardware Requirements

Development and model training will be conducted on a personal computer with a minimum of 8 GB RAM, a mid-range CPU (Intel Core i5 or AMD Ryzen 5 equivalent or better), and at least 50 GB of available local storage. No GPU acceleration is required, as all four classifier algorithms (Logistic Regression, KNN, Random Forest, XGBoost) are CPU-bound and the dataset sizes are compatible with CPU-based training. External storage (USB flash drive or cloud backup) of at least 16 GB will be used for backups. The deployed dashboard will run on Streamlit Cloud's shared virtual infrastructure, which provides sufficient resources for the dashboard's computational load.

Software Requirements

The development environment requires Python 3.10 or later (the primary implementation language), PostgreSQL 14 or later (the database engine for the existing datasets), and Git 2.x for version control. The production deployment environment is Streamlit Cloud (cloud-hosted, no local installation required).

Development Tools

Development tools include: Jupyter Notebooks (for reproducible EDA and modelling experiments); Visual Studio Code or PyCharm (for script development and dashboard code); pgAdmin or DBeaver (for PostgreSQL database access and query development); and GitHub (for version control and public repository hosting).

Python Libraries and Frameworks

Library / Framework	Version (Minimum)	Purpose
pandas	2.0+	Data manipulation, feature engineering, CSV I/O
numpy	1.24+	Numerical computation
scikit-learn	1.3+	LR, KNN, RF model training, cross-validation, metrics

Library / Framework	Version (Minimum)	Purpose
xgboost	1.7+	XGBoost model training and evaluation
shap	0.42+	SHAP feature importance and explainability
matplotlib	3.7+	Static EDA charts and result visualisations
seaborn	0.12+	Statistical EDA visualisations
plotly	5.14+	Interactive charts in Streamlit dashboard
streamlit	1.28+	Dashboard framework and deployment
sqlalchemy	2.0+	PostgreSQL connection from Python
psycopg2	2.9+	PostgreSQL adapter for Python
joblib	1.3+	Model serialisation and loading

Database

PostgreSQL 14 or later is the database management system. The existing developer-ecosystem-analytics repository provides the database schema and seed data. TechPulse accesses the database in read-only mode during the feature engineering pipeline execution. The database is hosted locally during development. No cloud database migration is required for the current project scope, although the connection string is configurable via environment variable to support future remote hosting.

Network and Internet Requirements

An internet connection is required for: pushing code to GitHub; deploying the dashboard to Streamlit Cloud; downloading Python package updates via pip; and accessing the deployed public dashboard URL. The system does not require a persistent internet connection for local development, model training, or local dashboard testing. The deployed dashboard requires a stable internet connection on the user side for access.

Cloud Services

Streamlit Cloud (free tier) is the only cloud service required. It provides: Python runtime hosting for the Streamlit application; a public HTTPS URL for dashboard access; and automatic redeployment from the GitHub repository on code push. If Streamlit Cloud becomes unavailable, a fallback local build will be maintained and documented, with Heroku noted as an alternative hosting option.

4.2.3 User Characteristics

User Role 1: Technology Decision-Makers (Primary Users)

Technology decision-makers include Chief Technology Officers, engineering managers, enterprise architects, and procurement officers within organisations. These users interact with the TechPulse Streamlit dashboard to support strategic decisions about technology adoption and risk management. They are expected to have a general understanding of software technology concepts but may not have deep machine learning expertise consequently, the dashboard must present predictions and explanations in plain language with appropriate visual support, without requiring the user to understand model internals. They are likely to use the dashboard infrequently but intensively, running multiple technology comparisons during a planning cycle. Their primary goals are to identify technologies at risk of decline before committing organisational resources, to compare risk scores across competing technology options, and to obtain evidence they can use to justify technology investment decisions to stakeholders. Their pain points include the current absence of reliable, data-driven forecasting tools and the cognitive overhead of manually synthesising community signals across multiple platforms.

User Role 2: Individual Developers (Primary Users)

Individual developers are professionals or students deciding where to invest their learning time and professional development effort. They have direct technical knowledge of the technologies they are querying and are comfortable using a web-based dashboard. They are likely to use the system periodically for example, when planning a learning roadmap or evaluating a new technology's long-term viability. Their primary goals are to identify which technologies have a growing or stable trajectory worth specialising in and to understand which specific signals are driving a technology's risk score. Their pain points include the difficulty of distinguishing genuine growth signals from hype and the absence of an authoritative data source for long-term technology viability.

User Role 3: Academic Researchers and Practitioners (Secondary Users)

Academic researchers and software analytics practitioners interact with TechPulse primarily through the open-source repository and the published dissertation rather than the dashboard alone. They are expected to have advanced technical knowledge, including familiarity with machine learning, feature engineering, and SHAP analysis. They may re-use the published

dataset, feature matrix, or model code in their own research. Their primary goal is to replicate, validate, or extend the predictive methodology developed in this project.

User Role 4: Project Supervisor (Administrative User)

The project supervisor (Dr. Rufus Gireka) interacts with the project through periodic progress reviews, dissertation chapter reviews, and the formal viva. The supervisor does not interact with the deployed dashboard as an end user but may access it for evaluation purposes. They require that the system is correctly scoped, methodologically sound, and that documentation meets academic standards.

4.2.4 Constraints

Technical Constraints: TechPulse is constrained to the six datasets already integrated in the developer-ecosystem-analytics repository. No new primary data collection is within scope. The feature engineering pipeline must derive all predictive signals from these existing datasets without requiring access to live external APIs. The four classifiers (Logistic Regression, KNN, Random Forest, XGBoost) are fixed for this project; deep learning approaches are out of scope. Model training must complete within a reasonable time on the development hardware (estimated at under two hours for the full training pipeline including hyperparameter tuning).

Budget Constraints: The project operates under a tightly constrained budget (estimated at KShs 8,000 total, including printing and contingency). All software tools and hosting services must be available at no cost or on free tiers. Any paid service or library is out of scope unless it can be demonstrated to be available without charge.

Time Constraints: The project must be completed within two academic semesters of 16 weeks each, totalling 32 weeks. All milestones defined in the project schedule (Appendix C) must be met within this window. Late delivery of any semester deliverable carries academic penalty risk.

Hardware Limitations: Model training is constrained to the capabilities of a personal computer with 8 GB RAM and a mid-range CPU. This limits the size of hyperparameter search spaces that can be explored during tuning. Datasets that exceed available RAM must be processed in chunks or sub-sampled.

Software Limitations: The Streamlit Cloud free tier imposes limits on concurrent users, memory allocation, and monthly active hours. Under high concurrent load, dashboard response

times may degrade. This is an accepted constraint for the academic project scope and is documented in the dashboard UI.

Legal and Ethical Constraints: Stack Overflow data is used under the Stack Exchange Creative Commons Attribution-ShareAlike licence. Fortune 500 and company profile data must be used in aggregate and anonymised form no individual-level personal data is processed by TechPulse. All predictions include confidence disclaimers to prevent misuse as sole decision-making tools.

Security Constraints: The deployed dashboard is a read-only public interface with no user authentication or data submission capability. No personal data is stored or transmitted. The PostgreSQL database is not exposed to the public internet; it is accessed only via the local development environment during the pipeline execution phase.

Academic Project Constraints: The system must be developed and documented to a standard suitable for an undergraduate final year project assessment. The dissertation must comply with all departmental formatting and submission requirements. The scope of the system must be achievable by a single student developer within the defined timeline.

4.3 Functional Requirements

This section defines all functional requirements for TechPulse. Requirements are organised by major system function and follow the standard FR specification format: Description, Input, Process, Output, Preconditions, Postconditions, Priority, Exceptions, Validation Rules, and Business Rules. Priority levels are: High (must be implemented for system viability), Medium (important for completeness), Low (desirable but deferrable).

FR-01 Dataset Ingestion and Validation

Description: The system shall ingest and validate all six source datasets from the PostgreSQL database prior to executing any feature engineering or modelling tasks. The ingestion process shall verify data integrity, check for unexpected null values, and report data quality statistics.

Input: PostgreSQL database connection parameters (host, port, database name, credentials via environment variable); names of the six source tables.

Process: The data pipeline connects to the PostgreSQL instance using SQLAlchemy, executes SELECT queries across all six tables, loads results into pandas DataFrames, validates each DataFrame for expected column presence, acceptable null rates (flagged if any key column exceeds 5% nulls), and minimum row counts (flagged if any table contains fewer than 50 rows). A data quality report is generated and logged.

Output: Six validated pandas DataFrames (one per dataset); a data quality report (JSON or plain text) summarising row counts, null rates, and validation pass/fail status per table.

Preconditions: The PostgreSQL database is running and accessible from the development environment; the six source tables exist and are populated with data from the developer-ecosystem-analytics repository.

Postconditions: All six DataFrames are loaded and validated; the data quality report is written to the output directory; the pipeline is ready to proceed to feature engineering.

Priority: High

Exceptions: Database connection failure: pipeline halts with a descriptive error message specifying the connection parameter that failed. Table not found: pipeline halts with the missing table name logged. Null rate exceeding threshold: pipeline continues but logs a warning with the affected column name and null percentage.

Validation Rules: All six expected table names must exist in the database. Each table must contain at least one non-null row for required key columns (e.g., technology name/tag, timestamp). Connection credentials must be provided via environment variable, not hard-coded.

Business Rules: No production data (personal or regulated) is ingested. All data is used for analytical purposes only under the applicable open-data licence terms.

FR-02 Exploratory Data Analysis (EDA)

Description: The system shall produce a comprehensive EDA report covering all six datasets, including univariate distribution analysis, bivariate correlation analysis between features and the target variable, time-series trend analysis for key activity metrics, and identification of class distribution for the target variable.

Input: The six validated DataFrames from FR-01.

Process: The EDA module computes: distribution statistics (mean, median, standard deviation, quartiles) for all numeric features; frequency distributions for categorical fields; Pearson and Spearman correlation matrices between numeric features; time-series plots of question volume, sentiment scores, and adoption rates over available time periods; and the distribution of the three target classes. All visualisations are saved as PNG files and compiled into a structured HTML/PDF EDA report.

Output: A set of EDA visualisation files (PNG format); an EDA report document summarising key findings; a correlation matrix CSV; and a class distribution summary.

Preconditions: FR-01 has completed successfully and all six DataFrames are available.

Postconditions: EDA report and all visualisation files are written to the output directory; key findings (e.g., top correlated features, class imbalance ratio) are logged for review.

Priority: High

Exceptions: Insufficient data for a given technology: the technology is flagged in the EDA report and excluded from the time-series analysis section. Empty technology categories: logged as a warning in the EDA report.

Validation Rules: All visualisations must include axis labels, titles, and source dataset attribution. Correlation values must be computed using at least 30 data points to be reported as statistically meaningful.

Business Rules: EDA findings inform feature selection decisions. Any feature with near-zero variance across the dataset will be flagged for potential exclusion from the modelling phase.

FR-03 Feature Engineering

Description: The system shall derive a predictive feature matrix from the six source datasets by computing or extending the seven defined predictive features: Technology Health Score, Growth Momentum Index, Question Quality Score, Company Diversity Score, Sentiment Delta, Adoption Velocity, and Community Decay Rate.

Input: The six validated DataFrames from FR-01; the definitions and SQL query logic for the four pre-existing derived metrics (queries 7, 9, 10, 12 from the developer-ecosystem-analytics repository).

Process: The feature engineering module executes or replicates the logic of existing SQL queries (7, 9, 10, 12) to compute the four pre-existing features. It then computes the three new features: Sentiment Delta (change in developer satisfaction score between the earliest and most recent survey period available); Adoption Velocity (number of new company adoptions per quarter, averaged over the trailing four quarters); and Community Decay Rate (percentage decline in Stack Overflow question volume over the trailing six months). All features are normalised to a common scale (min-max normalisation between 0 and 1) after computation. The resulting feature matrix is written to a CSV file.

Output: A feature matrix CSV file with one row per technology and seven feature columns plus a technology identifier column; a feature engineering log documenting the computation method and any missing-data decisions for each feature.

Preconditions: FR-01 has completed; FR-02 has completed (EDA findings have been reviewed and any near-zero-variance features have been identified).

Postconditions: The feature matrix CSV is written to the output directory; the feature engineering log is written; the pipeline is ready to proceed to target variable assignment and model training.

Priority: High

Exceptions: A technology has insufficient data to compute one or more features: the affected feature value is set to NaN and the technology is flagged. If more than three of the seven features are NaN for a technology, that technology is excluded from the modelling phase. SQL query execution failure: the pipeline halts with the query and error message logged.

Validation Rules: All seven feature values must be numeric. Normalised feature values must lie in the range [0, 1]. Any technology with a technology identifier containing null or empty string values is excluded.

Business Rules: Feature engineering logic must be fully documented and reproducible. Features computed from time-series data must use a consistent reference date (defined as the latest available data point in the database, not the current calendar date, to ensure reproducibility).

FR-04 Target Variable Assignment

Description: The system shall assign one of three class labels Growing, Stable, or Declining to each technology in the feature matrix based on the technology's Growth Momentum Index value and its Stack Overflow question volume trend direction over the trailing 12 months.

Input: The feature matrix from FR-03; the Growth Momentum Index values for each technology; the Stack Overflow question volume time series for each technology.

Process: The labelling module evaluates two criteria for each technology: (a) the Growth Momentum Index value, and (b) whether the 12-month question volume trend is positive, flat, or negative (determined by linear regression slope over the trailing 12 months of monthly question counts). Labels are assigned as follows: Growing if the Growth Momentum Index exceeds 70 AND the volume trend is positive; Declining if the Growth Momentum Index is below 40 OR the volume trend is consistently negative (negative slope with R^2 above 0.6); Stable for all other cases. The assigned labels are added to the feature matrix as the target column.

Output: The feature matrix CSV updated with a 'trajectory_label' column containing one of three string values: 'Growing', 'Stable', or 'Declining'; a labelling summary report showing the count and percentage of technologies in each class.

Preconditions: FR-03 has completed and the feature matrix CSV exists.

Postconditions: Every technology in the feature matrix has a non-null trajectory_label; the labelling summary is logged; the class distribution is passed to FR-02 output for documentation.

Priority: High

Exceptions: Technology has fewer than 12 months of question volume data: the volume trend criterion cannot be computed; the label is assigned based on the Momentum Index value alone, with a note appended in the labelling log. Boundary cases where Momentum Index is exactly 40 or 70: treated as Stable.

Validation Rules: Labels must be from the set {Growing, Stable, Declining}. No technology may carry more than one label. Technologies excluded in FR-03 due to missing data must also be absent from the labelled matrix.

Business Rules: The three-class labelling scheme is the ground-truth target for supervised classification. Label definitions are fixed for this project and documented in the dissertation to enable replication.

FR-05 Model Training Logistic Regression Baseline

Description: The system shall train a Logistic Regression classifier as the interpretable baseline model for the three-class technology trajectory classification task.

Input: The labelled feature matrix from FR-04; the train-test split produced by FR-07; scikit-learn LogisticRegression class.

Process: The training module fits a Logistic Regression model with L2 regularisation (default $C=1.0$) on the training split. The model is evaluated on the test split using accuracy, weighted F1-score, and per-class confusion matrix. Model coefficients are extracted and logged for interpretability. The trained model object is serialised to disk.

Output: A trained Logistic Regression model file (joblib format); classification report (precision, recall, F1 per class); confusion matrix; model coefficient table; training and test accuracy scores.

Preconditions: FR-04 has completed; FR-07 (train-test split) has been executed; scikit-learn is installed.

Postconditions: Logistic Regression model file is written to output directory; evaluation metrics are logged and stored in the results summary file.

Priority: High

Exceptions: Model fails to converge within the default maximum iterations: the solver is switched to 'saga' and max_iter is increased to 1000, with a warning logged. If convergence still fails: logged as a model failure and the model is marked as non-viable in the results comparison.

Validation Rules: All feature values must be numeric and free of NaN (imputed by median in FR-07). The multi-class strategy is 'ovr' (one-versus-rest). Regularisation strength C must remain at default unless the EDA reveals specific evidence for adjustment.

Business Rules: Logistic Regression serves as the interpretable baseline. Its coefficients provide the first-pass feature importance signal before SHAP analysis.

FR-06 Model Training K-Nearest Neighbours Baseline

Description: The system shall train a K-Nearest Neighbours (KNN) classifier as the non-parametric comparison baseline for the three-class classification task.

Input: The labelled feature matrix from FR-04; the train-test split from FR-07; scikit-learn KNeighborsClassifier.

Process: The training module trains a KNN classifier with k evaluated over the range [3, 5, 7, 9, 11] using 5-fold cross-validation on the training split. The k value producing the highest mean cross-validation accuracy is selected. The final model is evaluated on the test split. The trained model is serialised to disk.

Output: A trained KNN model file; classification report; confusion matrix; optimal k value selected; training and test accuracy scores.

Preconditions: FR-04 completed; FR-07 executed; scikit-learn installed.

Postconditions: KNN model file written; evaluation metrics logged.

Priority: Medium

Exceptions: All k values produce identical cross-validation scores: $k=5$ is used as the default. Dataset too small for KNN to produce meaningful results (fewer than 50 training samples): KNN is flagged as non-applicable and excluded from the final comparison.

Validation Rules: Features must be normalised before KNN training (ensured by FR-03). Distance metric is Euclidean (default). No data leakage k is selected using cross-validation on the training split only.

Business Rules: KNN is a non-parametric baseline. Its performance relative to Logistic Regression informs the complexity-performance trade-off analysis in the dissertation.

FR-07 Data Splitting and Cross-Validation Setup

Description: The system shall partition the labelled feature matrix into training and test sets and configure 5-fold stratified cross-validation for use by all four model training routines.

Input: The labelled feature matrix from FR-04.

Process: The data splitting module applies scikit-learn `train_test_split` with an 80/20 ratio and stratified sampling across the three target classes (`stratify=y`). The random state is fixed at 42 for reproducibility. Missing feature values (NaN) are imputed using the median of each feature computed on the training split only. The `StratifiedKFold` object ($k=5$, `shuffle=True`, `random_state=42`) is initialised and passed to all model training routines.

Output: Training feature matrix (80%); test feature matrix (20%); `StratifiedKFold` cross-validator object; imputed training and test feature matrices; class distribution report for both splits.

Preconditions: FR-04 completed.

Postconditions: Training and test splits are available in memory for all model training routines; the random state and split parameters are logged.

Priority: High

Exceptions: The labelled dataset contains fewer than 50 total samples: the system logs a critical warning and halts, as the split sizes would be insufficient for reliable evaluation. One class has fewer than 5 samples in total: the system logs a warning that the minority class may produce unreliable evaluation metrics.

Validation Rules: Stratified sampling must be used to preserve class proportions in both splits. The test set must not be used during any cross-validation or hyperparameter tuning step only during final evaluation. The fixed random state (42) must be applied consistently across all data splitting and model training steps.

Business Rules: Data leakage prevention is the highest-priority business rule for this requirement. No information from the test set may influence model training, feature scaling, or imputation computations.

FR-08 Model Training Random Forest

Description: The system shall train a Random Forest classifier with hyperparameter tuning as one of the two primary advanced classification models.

Input: Training feature matrix from FR-07; StratifiedKFold object from FR-07; scikit-learn RandomForestClassifier.

Process: The training module performs hyperparameter tuning using RandomizedSearchCV over a defined search space: `n_estimators` in [100, 200, 500]; `max_depth` in [None, 5, 10, 20]; `min_samples_split` in [2, 5, 10]; `min_samples_leaf` in [1, 2, 4]; `max_features` in ['sqrt', 'log2']. The search runs 20 iterations with 5-fold cross-validation, optimising for weighted F1-score. The best parameter set is used to train the final model on the full training split. The trained model is evaluated on the test split. Built-in feature importances are extracted and logged. The trained model is serialised to disk.

Output: Trained Random Forest model file; classification report; confusion matrix; best hyperparameter values; feature importance scores; training and test accuracy and F1 scores; ROC-AUC score (one-vs-rest).

Preconditions: FR-07 completed; scikit-learn installed.

Postconditions: Random Forest model file written; evaluation metrics and feature importances logged; results added to the model comparison table.

Priority: High

Exceptions: RandomizedSearchCV exhausts available RAM: the search space is reduced (`n_estimators` limited to [100, 200]) and the search is retried. Model training time exceeds 60

minutes: training is halted, the best partial result is saved, and the search is flagged as incomplete in the results log.

Validation Rules: The random state must be set to 42 in RandomizedSearchCV for reproducibility. Weighted F1-score (not accuracy) is the primary optimisation metric, to account for potential class imbalance. The test set must not be used in RandomizedSearchCV.

Business Rules: Random Forest is expected to be the primary model for SHAP analysis unless XGBoost outperforms it. Feature importances from Random Forest provide a secondary importance signal complementary to SHAP values.

FR-09 Model Training XGBoost

Description: The system shall train an XGBoost classifier with hyperparameter tuning as the second primary advanced classification model and the expected best-performing model.

Input: Training feature matrix from FR-07; StratifiedKFold object from FR-07; xgboost XGBClassifier.

Process: The training module performs hyperparameter tuning using RandomizedSearchCV over a defined search space: n_estimators in [100, 200, 500]; max_depth in [3, 5, 7]; learning_rate in [0.01, 0.05, 0.1, 0.2]; subsample in [0.6, 0.8, 1.0]; colsample_bytree in [0.6, 0.8, 1.0]; min_child_weight in [1, 3, 5]. The search runs 30 iterations with 5-fold cross-validation, optimising for weighted F1-score. The best parameter set is used to train the final model on the full training split. The trained model is evaluated on the test split. The model is serialised to disk.

Output: Trained XGBoost model file; classification report; confusion matrix; best hyperparameter values; training, validation, and test accuracy and F1 scores; ROC-AUC score.

Preconditions: FR-07 completed; xgboost library installed.

Postconditions: XGBoost model file written; evaluation metrics logged; results added to the model comparison table.

Priority: High

Exceptions: XGBoost training fails due to incompatible data types: feature matrix is cast to float32 and training is retried. Training time exceeds 90 minutes: search space is reduced (n_estimators limited to [100, 200]) and search retried.

Validation Rules: XGBoost requires numeric labels; the trajectory label column must be encoded as integers (0=Growing, 1=Stable, 2=Declining) before training. Label encoder mapping must be saved alongside the model for dashboard use.

Business Rules: XGBoost is the primary candidate for SHAP analysis and dashboard deployment. If Random Forest achieves a higher weighted F1-score on the test set, Random Forest is selected instead, and this decision is documented in the dissertation.

FR-10 Model Evaluation and Comparison

Description: The system shall produce a comprehensive model comparison report evaluating all four trained classifiers across all defined metrics and identifying the best-performing model for dashboard deployment.

Input: Trained model files from FR-05, FR-06, FR-08, FR-09; test feature matrix and test labels from FR-07.

Process: The evaluation module loads all four trained models, evaluates each on the held-out test set, and computes: overall accuracy; weighted F1-score; macro-averaged ROC-AUC (one-vs-rest, where applicable); per-class precision, recall, and F1-score; per-class confusion matrix. A comparison table is assembled. The model with the highest weighted F1-score on the test set is designated as the primary model for SHAP analysis and dashboard deployment.

Output: A model comparison table (CSV and rendered markdown); four confusion matrix visualisations; ROC curves for Random Forest and XGBoost; the name and file path of the selected best model; a written evaluation summary.

Preconditions: FR-05 through FR-09 have all completed successfully.

Postconditions: Best model is identified and its file path is recorded; the model comparison table and all visualisation files are written to the output directory.

Priority: High

Exceptions: Two models produce identical weighted F1-scores: the model with the higher macro-averaged ROC-AUC is selected; if still tied, XGBoost is preferred by default. A model file is missing or corrupt: that model is excluded from the comparison and flagged as failed.

Validation Rules: Evaluation must use only the held-out test set. The test set must not have been used in any cross-validation or tuning step. All metrics must be computed using scikit-learn's `classification_report` and `roc_auc_score` functions for consistency.

Business Rules: The selection of the best model is a one-time decision made at the end of the training phase. The selected model is not retrained after evaluation. The weighted F1-score is the primary selection criterion (not accuracy) to penalise models that misclassify minority classes.

FR-11 SHAP Feature Importance Analysis

Description: The system shall apply SHAP analysis to the best-performing model to generate per-feature importance scores, per-prediction explanations, and global feature importance rankings.

Input: The best-performing trained model file (from FR-10); the full feature matrix (training split); the SHAP library.

Process: The SHAP module initialises a SHAP TreeExplainer (for Random Forest or XGBoost) or LinearExplainer (for Logistic Regression) appropriate to the selected model type. It computes SHAP values for all samples in the training split. Global feature importance is derived as the mean absolute SHAP value per feature, producing a ranked list of the top 7 features. Per-prediction SHAP values are computed for a representative sample of 20 technologies (5 per quadrant of the Growing/Declining space) and stored as a JSON file for dashboard use. A SHAP summary plot (beeswarm) and bar chart are generated and saved as PNG files.

Output: SHAP explainer object (serialised for dashboard use); SHAP values array; global feature importance ranking (CSV); per-prediction SHAP JSON file; SHAP summary plot PNG; SHAP bar chart PNG.

Preconditions: FR-10 has completed and the best model file is identified.

Postconditions: SHAP explainer and value files are written to the output directory; the global feature importance ranking is logged; files are ready for dashboard integration.

Priority: High

Exceptions: SHAP TreeExplainer fails for the selected model type: the SHAP KernelExplainer is used as a fallback (slower but model-agnostic), with a warning logged. SHAP computation exceeds 30 minutes: the computation is limited to a 500-sample random subset of the training split, with this limitation noted in the dashboard UI.

Validation Rules: SHAP values must be computed on the training split only (not the test set) to avoid leakage of test-set information into the explainability outputs. The explainer object must be compatible with the serialised model file.

Business Rules: SHAP analysis is the primary mechanism for answering the research question about which features are the strongest predictors of technology decline. The global importance ranking must include all seven engineered features, even those with near-zero importance, to provide a complete answer.

FR-12 Dashboard Technology Search and Filtering

Description: The Streamlit dashboard shall provide a search interface allowing users to search for a specific technology by name and filter the technology list by category (languages, frameworks, databases, tools).

Input: User-entered text in the search field; selected category filter (dropdown); the full technology list loaded from the feature matrix CSV.

Process: On each user interaction, the dashboard filters the technology list using case-insensitive substring matching on the technology name field and applies the selected category filter. The filtered list is displayed in a data table below the search controls. The search updates dynamically on each keystroke using Streamlit's reactive execution model.

Output: A dynamically filtered technology list displayed in a Streamlit data table, with columns for technology name, category, trajectory label, and risk score.

Preconditions: The dashboard application is running and the feature matrix CSV and model files are loaded.

Postconditions: The filtered technology list reflects the current search text and category filter; the user can select a technology from the results to view its full prediction detail.

Priority: High

Exceptions: Search text produces no matching technologies: an informational message 'No technologies found matching your search criteria' is displayed. Feature matrix CSV is unavailable at dashboard startup: an error message is displayed and the dashboard does not proceed to prediction.

Validation Rules: Search is case-insensitive. Category filter options are derived dynamically from the unique values in the technology category column of the feature matrix. The search field accepts alphanumeric input and common special characters (e.g., '#', '+', '.') to support technology names such as C#, C++, and ASP.NET.

Business Rules: The search and filter UI is the primary entry point for all user interactions. It must be responsive and produce results within one second on the Streamlit Cloud deployment.

FR-13 Dashboard Technology Risk Score and Trajectory Prediction Display

Description: The dashboard shall display the trajectory prediction (Growing, Stable, or Declining) and a computed risk score for any technology selected by the user, along with a confidence indicator.

Input: The user-selected technology name; the trained model loaded at dashboard startup; the feature values for the selected technology from the feature matrix.

Process: The dashboard retrieves the feature vector for the selected technology from the loaded feature matrix, applies the trained model to produce a class prediction and class probability vector, computes the risk score as $(1 - \text{probability of Growing}) * 100$ (a score from 0 to 100 where 100 = highest risk), and displays the trajectory label, risk score, and a confidence indicator (Low if max class probability is below 0.6, Medium if 0.6 to 0.8, High if above 0.8).

Output: Displayed trajectory label (with colour coding: green for Growing, amber for Stable, red for Declining); displayed risk score (numeric, 0-100); displayed confidence indicator; a gauge or progress bar visualisation of the risk score.

Preconditions: A technology has been selected by the user from the search results; the model is loaded and the feature matrix is available.

Postconditions: The prediction, risk score, and confidence indicator are displayed on the technology detail panel; all three values are derived from the loaded trained model without re-training.

Priority: High

Exceptions: The selected technology is not in the feature matrix (e.g., a newly added technology not present at training time): a message 'Prediction unavailable insufficient historical data for this technology' is displayed. Model file is corrupt or missing: an error message is displayed and the dashboard prompts the user to contact the maintainer.

Validation Rules: Risk score must be expressed as an integer between 0 and 100. Confidence indicator labels must exactly match: 'Low Confidence', 'Medium Confidence', 'High Confidence'. Colour coding must use green (#2E9E6B), amber (#F4B942), and red (#D94F3D) consistently throughout the dashboard.

Business Rules: The risk score is a derived metric, not a direct model output. It is computed from the model's predicted class probability vector. The dashboard must display a disclaimer: 'TechPulse predictions are for informational purposes only and should not be used as the sole basis for technology investment decisions.'

FR-14 Dashboard Trend Charts and Historical Signal Visualisation

Description: The dashboard shall display interactive trend charts showing the historical trajectory of key signals for the selected technology over the available time range in the dataset.

Input: The selected technology name; the Stack Overflow question volume time series from the source database; the sentiment score time series; the company adoption count over time.

Process: The dashboard retrieves and plots three interactive Plotly charts: (a) monthly Stack Overflow question volume over time (line chart); (b) developer sentiment score over available survey periods (line or bar chart); (c) company adoption count per year (bar chart). Charts are rendered using Plotly within Streamlit's `st.plotly_chart` component.

Output: Three interactive Plotly charts displayed in the dashboard technology detail panel, each with labelled axes, a title, and a hover tooltip showing the exact value at each data point.

Preconditions: A technology has been selected; the source data time series are available in the loaded feature matrix or a supplementary data file.

Postconditions: All three trend charts are rendered and interactive; the user can hover over data points to see exact values.

Priority: High

Exceptions: Fewer than three data points are available for any chart: the chart is replaced by a message 'Insufficient data to plot trend for this signal'. The plotly library raises a rendering error: the chart section is replaced by a static fallback table of values.

Validation Rules: Chart axes must be labelled. Chart titles must identify the technology name and the signal being plotted. Time axis must display dates in ISO 8601 format (YYYY-MM). The Y-axis scale must start at zero for volume and adoption charts.

Business Rules: Trend charts are a core UX feature for helping users understand the basis for predictions. They must be rendered within two seconds of technology selection on the Streamlit Cloud deployment.

FR-15 Dashboard SHAP Feature Importance Viewer

Description: The dashboard shall display a SHAP feature importance visualisation for the selected technology, showing which features contributed most positively or negatively to the predicted trajectory label.

Input: The selected technology name; the pre-computed per-prediction SHAP values JSON from FR-11 (for the representative sample) or on-demand SHAP computation for technologies outside the sample.

Process: The dashboard retrieves the SHAP values for the selected technology from the pre-computed JSON file if available. If not available, the loaded SHAP explainer object is used to compute SHAP values on-demand for the selected technology's feature vector. A horizontal bar chart is rendered showing each feature's SHAP value, coloured red for positive contributions to a high-risk prediction and blue for negative contributions (risk-reducing). Feature names and their human-readable descriptions are displayed on the Y-axis.

Output: A horizontal SHAP bar chart displayed in the technology detail panel, with feature names on the Y-axis, SHAP values on the X-axis, and a vertical zero line. A text summary beneath the chart states the top two positive and top two negative contributing features in plain English.

Preconditions: A technology has been selected; the SHAP explainer object is loaded; pre-computed or on-demand SHAP values are available.

Postconditions: SHAP chart is rendered; the plain-English summary is displayed.

Priority: High

Exceptions: On-demand SHAP computation exceeds 10 seconds: a loading spinner is shown. SHAP computation fails: the SHAP viewer section is replaced by a message 'Feature explanation unavailable for this technology'. SHAP library is not installed: the dashboard displays a warning and the SHAP section is hidden.

Validation Rules: SHAP bar chart must use a diverging colour scheme. Feature labels must match the human-readable feature descriptions defined in the data dictionary. The zero line must be visible and labelled.

Business Rules: The SHAP viewer is a key transparency and trust-building feature of TechPulse. The plain-English summary must use non-technical language accessible to non-ML users (e.g., 'Stack Overflow question quality is the strongest indicator driving the Growing prediction for Python').

FR-16 Dashboard Enterprise Adoption Heatmap

Description: The dashboard shall display a heatmap or table showing which Fortune 500 companies in the dataset adopt the selected technology, segmented by industry sector.

Input: The selected technology name; the Fortune 500 Tech Stacks and Company Profiles datasets loaded at dashboard startup.

Process: The dashboard filters the Fortune 500 dataset for rows where the technology name matches the selected technology, joins with the Company Profiles table on company identifier to retrieve sector information, and renders a heatmap or pivot table showing adoption count by sector. If fewer than three adopting companies are found, a list of company names is displayed instead of a heatmap.

Output: A sector heatmap or adoption count table displayed in the dashboard; if applicable, a list of adopting company names.

Preconditions: A technology has been selected; the Fortune 500 and Company Profiles datasets are loaded at dashboard startup.

Postconditions: The enterprise adoption section of the technology detail panel is populated.

Priority: Medium

Exceptions: No Fortune 500 companies in the dataset adopt the selected technology: a message 'No Fortune 500 adoption data available for this technology' is displayed. The sector join produces null sector values for some companies: those companies are categorised under 'Unknown Sector'.

Validation Rules: Company names and sectors must be displayed consistently with the values in the Company Profiles source dataset. No individual company financial data or confidential information is displayed only sector-level counts and company names already present in the publicly sourced dataset.

Business Rules: The enterprise adoption heatmap differentiates TechPulse from community-signal-only tools and directly addresses the research question about whether Fortune 500 adoption correlates with technology longevity.

FR-17 Dashboard Global Technology Risk Rankings

Description: The dashboard shall provide a global overview page displaying a ranked list of all technologies in the feature matrix, ordered by risk score from highest to lowest, with filtering by category and trajectory label.

Input: The full feature matrix CSV loaded at startup; category and trajectory label filter selections from the user.

Process: The dashboard renders a Streamlit data table or Plotly bar chart displaying all technologies in the feature matrix, ranked by risk score (highest risk first). Category and trajectory label filters narrow the displayed list. A download button allows the user to export the filtered list as a CSV file.

Output: A ranked data table or bar chart of all technologies with columns: Technology Name, Category, Trajectory Label, Risk Score, Confidence Level. A CSV download button.

Preconditions: Dashboard is running and the feature matrix is loaded.

Postconditions: The global rankings page is rendered; filtering and export are functional.

Priority: Medium

Exceptions: Feature matrix CSV is empty or contains only one technology: a message 'Insufficient technologies in dataset for ranking' is displayed. CSV export fails: a toast error message is displayed.

Validation Rules: Risk scores must be displayed as integers 0-100. The data table must support column sorting by the user. Category filter must include an 'All Categories' default option.

Business Rules: The global rankings page is the primary entry point for users who have not yet identified a specific technology of interest. It must load within two seconds of the dashboard startup.

FR-18 Dashboard Model Performance Summary Page

Description: The dashboard shall provide a model performance page displaying the evaluation metrics of all four trained classifiers, the model comparison table, and the confusion matrix for the best-performing model.

Input: The model comparison table CSV and confusion matrix PNG files from FR-10, loaded at dashboard startup.

Process: The dashboard renders a static model performance page showing: the model comparison table (accuracy, F1, ROC-AUC for all four models); the confusion matrix for the selected best model; a brief plain-English explanation of what the metrics mean.

Output: A rendered model comparison table; a confusion matrix visualisation; a plain-English metrics explanation.

Preconditions: Dashboard is running; model comparison files are available.

Postconditions: Model performance page is rendered and accessible via the dashboard navigation.

Priority: Medium

Exceptions: A model comparison file is missing: the affected row in the comparison table is displayed with 'N/A' values and a note that the model training failed.

Validation Rules: All metric values must be rounded to four decimal places. The confusion matrix must use a colour scale consistent with the dashboard's colour scheme.

Business Rules: Transparency about model performance builds user trust and is required for academic assessment. The model performance page must clearly state which model was selected for dashboard use and why.

FR-19 Dashboard Disclaimer and Documentation Page

Description: The dashboard shall include an About/Documentation page that provides: a description of TechPulse's purpose and methodology; a description of the data sources; the model limitation disclaimer; and links to the GitHub repository and dissertation.

Input: No user input required; static content loaded at dashboard startup.

Process: The dashboard renders a static documentation page with: project description; data source descriptions and licence attributions; model limitation statement; link to the GitHub repository; link to the dissertation PDF (if publicly available); contact information (student name and ORCID).

Output: A rendered documentation page accessible via the dashboard navigation.

Preconditions: Dashboard is running.

Postconditions: Documentation page is accessible.

Priority: Medium

Exceptions: No exceptions applicable.

Validation Rules: All external links must open in a new browser tab. The Stack Exchange Creative Commons licence must be explicitly attributed for Stack Overflow data. The disclaimer must appear on both the documentation page and the technology detail page.

Business Rules: The documentation page fulfils both the academic requirement for source attribution and the ethical obligation to communicate model limitations to users.

FR-20 Dissertation Documentation

Description: The system shall be documented in a formal academic dissertation covering Introduction, Literature Review, Methodology, Results, Discussion, and Recommendations chapters, meeting all BSc. Data Science departmental requirements.

Input: All project outputs: EDA reports, model evaluation results, SHAP analysis outputs, dashboard screenshots, and the project schedule.

Process: The student drafts and revises dissertation chapters following the schedule defined in Appendix C (Semester 2, Weeks 6-13). Each chapter is reviewed by the supervisor before finalisation. The complete dissertation is submitted in bound format by Week 14 of Semester 2.

Output: A complete, bound dissertation document submitted for academic assessment; a digital copy uploaded to the institutional submission portal.

Preconditions: All model training and evaluation activities (FR-01 through FR-11) have been completed; all dashboard features (FR-12 through FR-19) have been implemented and deployed.

Postconditions: Dissertation is submitted by the Week 14 deadline.

Priority: High

Exceptions: Supervisor review feedback requires significant revision: additional revision time must be allocated within the Week 12-13 buffer period. Submission portal is unavailable: the institution's alternative submission procedure is followed.

Validation Rules: Dissertation must comply with all institutional formatting requirements (font, margins, referencing style, word count). All figures must be captioned. All tables must be numbered. All sources must be cited using the institutional referencing style.

Business Rules: The dissertation is the primary academic output of the project. All system components are in service of producing a dissertation of distinction-level quality.

4.3.1 Use Case Diagram Description

The TechPulse system supports the following actors and use cases, described in UML-ready terms for conversion to a formal use case diagram.

Actors

Primary Actors: (1) Technology Decision-Maker a human user who interacts with the Streamlit dashboard to query technology risk scores; (2) Individual Developer a human user with the same dashboard interactions as the Decision-Maker, distinguished by their personal development goals; (3) Data Scientist / Project Developer (Tony Kenga) the human actor who executes the data pipeline, model training routines, and dashboard deployment scripts.

Secondary Actors: (4) PostgreSQL Database an external system that provides read-only access to the six source datasets; (5) Streamlit Cloud an external cloud platform that hosts the deployed dashboard; (6) GitHub an external platform for version control and public repository hosting.

Major Use Cases

UC-01: Search for Technology Actor: Technology Decision-Maker or Developer. The actor enters a technology name or selects a category filter. The system returns a filtered list of matching technologies with their trajectory labels and risk scores.

UC-02: View Technology Prediction Detail Actor: Technology Decision-Maker or Developer. The actor selects a technology from the search results. The system displays the trajectory prediction, risk score, confidence indicator, trend charts, SHAP feature importance, and enterprise adoption data.

UC-03: Browse Global Risk Rankings Actor: Technology Decision-Maker or Developer. The actor navigates to the global rankings page. The system displays all technologies ranked by risk score with category and label filters.

UC-04: Export Risk Rankings Actor: Technology Decision-Maker. The actor clicks the download button on the global rankings page. The system exports the current filtered ranking as a CSV file.

UC-05: View Model Performance Actor: Technology Decision-Maker or Academic Researcher. The actor navigates to the model performance page. The system displays the four-model comparison table and best-model confusion matrix.

UC-06: Execute Data Pipeline Actor: Data Scientist. The actor runs the Python data pipeline script. The system ingests data (FR-01), performs EDA (FR-02), engineers features (FR-03), assigns labels (FR-04), trains models (FR-05 through FR-09), evaluates and selects the best model (FR-10), and runs SHAP analysis (FR-11).

UC-07: Deploy Dashboard Actor: Data Scientist. The actor pushes code to the GitHub repository. Streamlit Cloud automatically deploys the updated dashboard to the public URL.

4.4 Non-Functional Requirements

4.4.1 Performance

NFR-P-01: Dashboard Search Response Time. The technology search and filter operation (FR-12) must return filtered results within one second (1s) of user input on the Streamlit Cloud deployment, measured under normal load (fewer than 10 concurrent users). This target accounts for Streamlit's reactive re-execution model and the expected feature matrix size of under 500 rows.

NFR-P-02: Prediction and Display Latency. The technology detail panel (FR-13 through FR-16) must fully render including prediction, risk score, trend charts, and pre-computed SHAP values within two seconds (2s) of technology selection under normal load. On-demand SHAP computation (for technologies outside the pre-computed sample) may take up to ten seconds (10s), during which a loading indicator must be displayed.

NFR-P-03: Model Training Throughput. The full model training pipeline (FR-05 through FR-09, including hyperparameter tuning) must complete within four hours (4h) on the specified development hardware (8 GB RAM, mid-range CPU). XGBoost and Random Forest tuning are the most computationally intensive steps; search space definitions must be calibrated to meet this target.

NFR-P-04: Data Pipeline Execution Time. The data ingestion and feature engineering pipeline (FR-01 through FR-04) must complete within 30 minutes on the development hardware, assuming the PostgreSQL database is locally accessible.

4.4.2 Scalability

NFR-SC-01: Dataset Scalability. The feature engineering and modelling pipeline must be capable of processing feature matrices containing up to 1,000 technology records without requiring architectural changes. The current expected feature matrix size is approximately 50 to 200 technologies, but the pipeline must not hard-code assumptions about dataset size.

NFR-SC-02: Dashboard User Scalability. The Streamlit dashboard is expected to serve up to 10 concurrent users under normal operation. The Streamlit Cloud free tier supports this load. If concurrent usage exceeds this threshold, the deployment plan will be reviewed; however, exceeding this threshold is not expected during the academic project lifecycle.

4.4.3 Security

NFR-SE-01: No User Authentication Required. The Streamlit dashboard is a read-only public interface. No user registration, login, session management, or personal data collection is implemented or required. This design decision eliminates an entire class of authentication-related security vulnerabilities.

NFR-SE-02: Database Credential Security. PostgreSQL connection credentials must never be hard-coded in source files. They must be provided exclusively via environment variables (e.g., DATABASE_URL) and must not be committed to the GitHub repository. A .gitignore file must be configured to exclude all .env files.

NFR-SE-03: Dependency Security. All Python dependencies must be pinned to specific versions in a requirements.txt file. Dependency versions must be reviewed for known vulnerabilities using pip-audit or equivalent tooling before deployment.

NFR-SE-04: HTTPS Enforcement. The deployed Streamlit Cloud dashboard is served exclusively over HTTPS. No HTTP access is supported. This is enforced by the Streamlit Cloud platform and requires no additional configuration by the developer.

4.4.4 Availability

NFR-AV-01: Dashboard Uptime Target. The deployed TechPulse dashboard shall maintain a target availability of 95% during the academic assessment period (Semester 2, Weeks 1 through 16). Downtime due to Streamlit Cloud platform maintenance is excluded from this calculation.

NFR-AV-02: Local Fallback. A locally executable version of the dashboard must be maintained and documented at all times during development and assessment. If Streamlit Cloud becomes unavailable, the local version must be deployable within 30 minutes on the development machine.

4.4.5 Reliability

NFR-RL-01: Pipeline Idempotency. The data pipeline (FR-01 through FR-04) must be idempotent running the pipeline multiple times with the same input data must produce identical

output files. This is enforced by fixing the random state (42) in all stochastic operations (train-test split, cross-validation, model training).

NFR-RL-02: Error Handling. All pipeline steps must implement try-except error handling for I/O operations, database queries, and model training calls. Errors must be logged with sufficient detail (step name, error type, error message, stack trace) to enable diagnosis and remediation. The pipeline must not silently swallow exceptions.

NFR-RL-03: Model Versioning. Each trained model file must include a version timestamp in its filename (e.g., `techpulse_rf_model_20260620.joblib`). The dashboard must load the most recent model version by date, with a fallback to a hardcoded default model filename if no timestamped files are found.

4.4.6 Maintainability

NFR-MA-01: Code Quality. All Python code must follow PEP 8 style guidelines. Functions must be documented with docstrings specifying parameters, return values, and exceptions raised. The codebase must achieve a pylint score of at least 7.0 out of 10.

NFR-MA-02: Modularity. The pipeline code must be organised into separate, single-responsibility Python modules: `data_ingestion.py`, `eda.py`, `feature_engineering.py`, `labelling.py`, `model_training.py`, `evaluation.py`, `shap_analysis.py`, and `dashboard.py` (the Streamlit app). This structure enables individual components to be tested, replaced, or extended independently.

NFR-MA-03: Reproducibility. All random operations must use a fixed random state (42). All data processing steps must be logged with sufficient detail that a third party can reproduce the results from the raw datasets and source code alone.

4.4.7 Usability

NFR-US-01: Non-Expert Accessibility. The dashboard UI must be comprehensible to a user with no machine learning background. All technical terms (SHAP values, F1-score, ROC-AUC) must be accompanied by plain-English tooltips or explanatory text. The technology trajectory labels (Growing, Stable, Declining) must be the primary communication channel, supported by not replaced by technical metrics.

NFR-US-02: Colour Accessibility. The dashboard colour scheme must meet WCAG 2.1 AA contrast requirements. Colour-coded trajectory labels (green, amber, red) must also be accompanied by text labels so that the information is accessible to users with colour vision deficiencies.

NFR-US-03: Loading Feedback. Any dashboard operation expected to take more than one second must display a loading spinner or progress indicator. The user must never be presented with a blank or frozen interface without visual feedback that the system is working.

4.4.8 Data Integrity

NFR-DI-01: Source Data Immutability. TechPulse accesses the PostgreSQL database in read-only mode. No INSERT, UPDATE, or DELETE operations are executed by TechPulse against any source table. This is enforced at the application level (read-only SQL queries) and documented in the system design.

NFR-DI-02: Feature Matrix Consistency. The feature matrix CSV produced by the feature engineering pipeline must be deterministic given the same source data. Any non-deterministic operation (random sampling, random splits) must use the fixed random state. The feature matrix checksum (MD5 hash) must be logged at the end of each pipeline run.

NFR-DI-03: Label Consistency. The target variable assignment logic (FR-04) must be deterministic and documented. The same technology with the same source data values must always receive the same label across pipeline runs.

4.4.9 Privacy and Compliance

NFR-PC-01: No Personal Data Processing. TechPulse does not process, store, or transmit any personal data about individual developers, employees, or users. All analytical data is aggregated at the technology level. This design ensures that TechPulse does not trigger obligations under personal data protection regulations.

NFR-PC-02: Data Licence Compliance. All source datasets must be used in accordance with their applicable licences: Stack Overflow data under the Stack Exchange Creative Commons Attribution-ShareAlike 4.0 International licence; all other datasets under their respective open-

data licences as documented in the existing developer-ecosystem-analytics repository. The dashboard documentation page must display explicit licence attributions.

NFR-PC-03: Model Disclaimer. All prediction outputs must be accompanied by a disclaimer stating that TechPulse predictions are for informational purposes only and must not be used as the sole basis for technology investment, hiring, or strategic decisions. This disclaimer must be visible on all prediction-displaying pages without the user needing to scroll.

4.4.10 Auditability and Logging

NFR-AL-01: Pipeline Execution Logging. The data pipeline must produce a timestamped execution log (plain text file) for each run, recording: start and end time of each step; row counts and null rates at each stage; random state values used; model selection decision and reason; and any warnings or errors encountered. Logs must be stored in a designated `/logs/` directory and not committed to the public GitHub repository.

NFR-AL-02: Model Audit Trail. For each training run, a JSON file must be produced recording: the model type and hyperparameters used; the train-test split ratio and random state; the evaluation metrics on the test set; and the SHAP top-5 feature importance values. This audit trail enables retrospective inspection of any model version.

4.5 External Requirements

4.5.1 User Interfaces

UI-01: Dashboard Home Page / Technology Search Page

Purpose: Provide the primary entry point for all user interactions with TechPulse. **Displayed Information:** TechPulse logo and system name; a navigation bar with links to Home, Global Rankings, Model Performance, and About pages; a text search field labelled 'Search Technology'; a category dropdown filter with options All, Languages, Frameworks, Databases, Tools, Other; a results table displaying Technology Name, Category, Trajectory Label (colour-coded), and Risk Score (0-100). **Buttons:** None on the search page itself; the results table rows are clickable to navigate to the technology detail panel. **Input Fields:** Search text field (maximum 100 characters, alphanumeric plus common punctuation); category dropdown (single-select). **Validation:** Search field accepts all characters; empty search displays all technologies (up to 200 rows). **Navigation:** Results table row click navigates to the technology detail panel for the selected technology. **Error Messages:** 'No technologies found matching your search criteria' displayed below the search field if no results match. **User Interactions:** Real-time filtering on keystroke; dropdown selection triggers immediate filter update.

UI-02: Technology Detail Panel

Purpose: Display the full prediction detail and supporting evidence for a selected technology. **Displayed Information:** Technology name (large header); trajectory label badge (colour-coded: green/amber/red); risk score gauge (0-100); confidence indicator; tabs for Overview, Trends, SHAP Analysis, Enterprise Adoption; feature value table; data recency note (date of most recent data point used in prediction). **Buttons:** Back to Search Results; Download Feature Data (CSV). **Input Fields:** None; this is a display panel. **Validation:** N/A. **Navigation:** Back button returns to search results with the previous search state preserved. **Error Messages:** 'Prediction unavailable insufficient historical data' if the technology is not in the feature matrix; 'Feature explanation unavailable' if SHAP computation fails; 'No trend data available' if fewer than three data points exist. **User Interactions:** Tab navigation between Overview, Trends, SHAP Analysis, and Enterprise Adoption sections. **Disclaimer:** 'TechPulse predictions are for informational purposes only' displayed below the risk score gauge.

UI-03: Global Risk Rankings Page

Purpose: Provide a ranked overview of all technologies in the system by risk score. **Displayed Information:** A sortable data table with columns: Rank, Technology Name, Category,

Trajectory Label, Risk Score, Confidence Level. Filtering controls for category and trajectory label. Row count indicator. Buttons: Download CSV (exports the current filtered ranking). Input Fields: Category filter dropdown; Trajectory Label filter dropdown (Growing, Stable, Declining, All). Validation: Filters are applied server-side by Streamlit reactive execution. Navigation: Table rows are clickable to navigate to the technology detail panel. Error Messages: 'No data available' if the feature matrix fails to load.

UI-04: Model Performance Page

Purpose: Display the model evaluation results for all four trained classifiers. Displayed Information: Model comparison table (Model Name, Accuracy, Weighted F1, ROC-AUC, Status); best model indicator; confusion matrix heatmap for the best model; brief explanation of each metric in plain English. Buttons: None. Input Fields: None. Navigation: Accessible via navigation bar. Error Messages: 'Model evaluation data unavailable' if the model comparison CSV is missing.

UI-05: About / Documentation Page

Purpose: Provide project context, data source attributions, model limitations disclaimer, and contact information. Displayed Information: Project description; data source descriptions with licence attributions; model limitations and disclaimer; link to GitHub repository (opens in new tab); link to dissertation PDF (if available); student name, ORCID, and academic year. Buttons: GitHub Repository link; Dissertation PDF link. Input Fields: None. Navigation: Accessible via navigation bar.

4.5.2 Software Interfaces

PostgreSQL 14+ Database Interface

TechPulse interacts with the PostgreSQL database exclusively as a read-only consumer. The interface is implemented via the SQLAlchemy ORM (version 2.0+) and the psycopg2 adapter (version 2.9+). The connection string is provided via the DATABASE_URL environment variable. All queries are executed as parameterised statements to prevent SQL injection, although no user input reaches the database query layer in the current system design. The database schema comprises six tables as defined in the developer-ecosystem-analytics repository documentation. TechPulse does not modify the schema, execute DDL statements, or create new tables.

Streamlit Framework Interface

The dashboard is implemented as a single-page Streamlit application (version 1.28+) with multi-page navigation implemented using Streamlit's `st.navigation` or `st.Page` API. The application loads the feature matrix CSV and trained model files from the local filesystem at startup using `joblib` and `pandas`. Interactive components (search fields, dropdowns, data tables, charts) use Streamlit's built-in widget API. Chart rendering uses Plotly (via `st.plotly_chart`) for interactivity and Matplotlib for static figures.

Streamlit Cloud Deployment Interface

The dashboard is deployed to Streamlit Cloud by connecting the GitHub repository to a Streamlit Cloud workspace. Deployment is triggered automatically on each push to the main branch. The Streamlit Cloud platform reads the `requirements.txt` file and installs dependencies in the cloud Python environment. Secrets (database credentials, if required) are configured via the Streamlit Cloud Secrets Management UI and are not stored in the repository.

GitHub Version Control Interface

The project codebase, including the data pipeline scripts, dashboard code, `requirements.txt`, and documentation files, is managed in a public GitHub repository (github.com/Tony405-spec/developer-ecosystem-analytics or a fork thereof). Git is used for version control, with commits made at the end of each working session and at the completion of each milestone deliverable.

SHAP Library Interface

The SHAP library (version 0.42+) is used to generate `TreeExplainer` or `LinearExplainer` objects compatible with the selected best model. The SHAP explainer is initialised on the training data and serialised using `joblib`. The dashboard loads the serialised explainer at startup and invokes it on individual feature vectors to produce per-prediction SHAP values displayed in the UI.

4.5.3 Hardware Interfaces

Development Computer

TechPulse is developed and trained on a personal computer with a minimum specification of: Intel Core i5 or AMD Ryzen 5 processor (or equivalent); 8 GB RAM (16 GB recommended for comfortable hyperparameter tuning); 50 GB available disk space (for dataset files, model files, logs, and dissertation); standard keyboard and display. No specialised hardware (GPU,

FPGA, custom accelerator) is required or used. All computation is CPU-bound and compatible with standard consumer hardware.

End-User Device

End users access the TechPulse dashboard through a modern web browser (Chrome, Firefox, Safari, or Edge, version released within the past two years) on any device with internet connectivity. The dashboard is designed for desktop and laptop display resolutions of 1280x720 pixels or higher. Mobile device compatibility is a nice-to-have but is not formally required for the academic project scope; Streamlit's responsive layout provides basic mobile support without additional configuration.

Network Hardware

No specialised network hardware is required. A standard broadband internet connection (minimum 10 Mbps download) is sufficient for: pushing code to GitHub; deploying to Streamlit Cloud; and accessing the deployed dashboard. The PostgreSQL database is accessed via localhost during development; no network hardware beyond a standard network interface card is required.

References

- Borges, H., & Valente, M. T. (2018). What's in a GitHub star? Understanding repository starring practices in a social coding platform. *Journal of Systems and Software*, 146, 112–129. <https://doi.org/10.1016/j.jss.2018.09.016>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. <https://doi.org/10.1145/2939672.2939785>
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems* (Vol. 30, pp. 4765–4774). Curran Associates.
- Mens, T., & Goeminne, M. (2012). Analysing the evolution of social coding platforms. *Science of Computer Programming*, 78(8), 986–1005. <https://doi.org/10.1016/j.scico.2012.04.008>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Ray, B., Posnett, D., Filkov, V., & Devanbu, P. (2014). A large scale study of programming languages and code quality in GitHub. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (pp. 155–165). ACM. <https://doi.org/10.1145/2635868.2635922>
- Stack Exchange, Inc. (2024). Stack Overflow developer survey 2024. Stack Exchange, Inc. <https://survey.stackoverflow.co/2024/>
- Streamlit Inc. (2023). Streamlit documentation (Version 1.28). <https://docs.streamlit.io/>
- Syer, M. D., Adams, B., Hassan, A. E., & Godfrey, M. W. (2013). Revisiting prior empirical findings for agile: An analysis of the RapidSMS open-source project. In *Proceedings of the 2013 Conference of the Centre for Advanced Studies on Collaborative Research* (pp. 51–65). IBM Corp.
- IEEE Computer Society. (2011). IEEE standard glossary of software engineering terminology (IEEE Std 610.12-1990, reaffirmed 2002). IEEE. <https://standards.ieee.org/>

Wiegers, K., & Beatty, J. (2013). Software requirements (3rd ed.). Microsoft Press.